Data Logging using AdvantageScope
Written by Ridham

By this point you have created many complex programs. However, debugging and testing must still be covered. We will now go over data logging with Advantage Scope. Advantage Scope is a powerful tool that allows us to easily graph our logs accurately. It also allows us to see where the robot thinks it is through 2d fields. Today, we will be implementing Advantage Scope to use a 2d field that lets us see where the romi is. Clone your github branch of Romi 2026 if not already done and open it.

First, select Advantage Scope as your start tool. Then connect to your romi and simulate. Then in Advantage Scope look at the top left, click file, and under connect to simulator, choose Network Tables 4. This will let your Advantage Scope use any data sent to the simulator, such as anything that is sent to the smart dashboard. To reconnect to the simulator, you can also press ctrl + shift + k.

In order to get the robot's position to be visualized on the field, you need to log the position as a field 2d. First, create a new field 2d. Your code should be similar to the image below. You will also need to import edu.wpi.first.wpilibj.smartdashboard.Field2d

```
33  public class RomiDrivetrain extends SubsystemBase implements Loggable {
34    // DataLog variables
35    private int logRotationKey;         // key for the logging cycle for this subsystem
36    private boolean fastLogging = false; // true is enabled to run every cycle; false follows normal logging cycles
37    private final DataLog log = DataLogManager.getLog();
38    private final StructLogEntry<Pose2d> dLogOdometryPose2D = StructLogEntry.create(log, name: "/RomiDrivetrain/curPose2d", Pose2d.struct);
39    private final DoubleLogEntry dLogLeftPercent = new DoubleLogEntry(log, name: "/RomiDrivetrain/LeftPercent");
40    private final DoubleLogEntry dLogRightPercent = new DoubleLogEntry(log, name: "/RomiDrivetrain/RightPercent");
41    private final DoubleLogEntry dLogLeftDistInches = new DoubleLogEntry(log, name: "/RomiDrivetrain/LeftDistInches");
42    private final DoubleLogEntry dLogRightDistInches = new DoubleLogEntry(log, name: "/RomiDrivetrain/RightDistInches");
43    private final DoubleLogEntry dLogLeftVelocityIPS = new DoubleLogEntry(log, name: "/RomiDrivetrain/LeftVelocityIPS");
44    private final DoubleLogEntry dLogRightVelocityIPS = new DoubleLogEntry(log, name: "/RomiDrivetrain/RightVelocityIPS");
45    private final DoubleLogEntry dLogGyroAngle = new DoubleLogEntry(log, name: "/RomiDrivetrain/GyroAngle");
46    private final Field2d field = new Field2d();
47
48    // The Romi has the left and right motors set to
49    // PWM channels 0 and 1 respectively
50    private final Spark m_leftMotor = new Spark(leftDriveMotor);
51    private final Spark m_rightMotor = new Spark(rightDriveMotor);
```

Next, in the constructor for your romi drivetrain, initialize the smart dashboard with the field variable. Make sure your code is similar to the below image.

```
public RomiDrivetrain() {
  logRotationKey = DataLogUtil.allocateLogRotation();     // Get log rotation for this subsystem
  SmartDashboard.putData(field);

  // Initialize the drivetrain hardware
  // Use inches as unit for encoder distances
  m_leftEncoder.setDistancePerPulse((Math.PI * kWheelDiameterInch) / kCountsPerRevolution);
  m_rightEncoder.setDistancePerPulse((Math.PI * kWheelDiameterInch) / kCountsPerRevolution);
  zeroEncoders();
  zeroGyroRotation();

  // Invert right side since motor is flipped
  m_rightMotor.setInverted(isInverted: true);
```

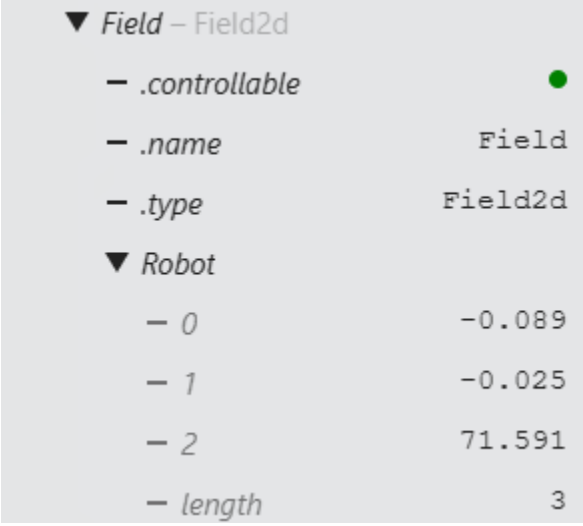
Afterwards, update the field variable in periodic. Reference the code below as an example.

```java
@Override
public void periodic() {
  // This method will be called once per scheduler run

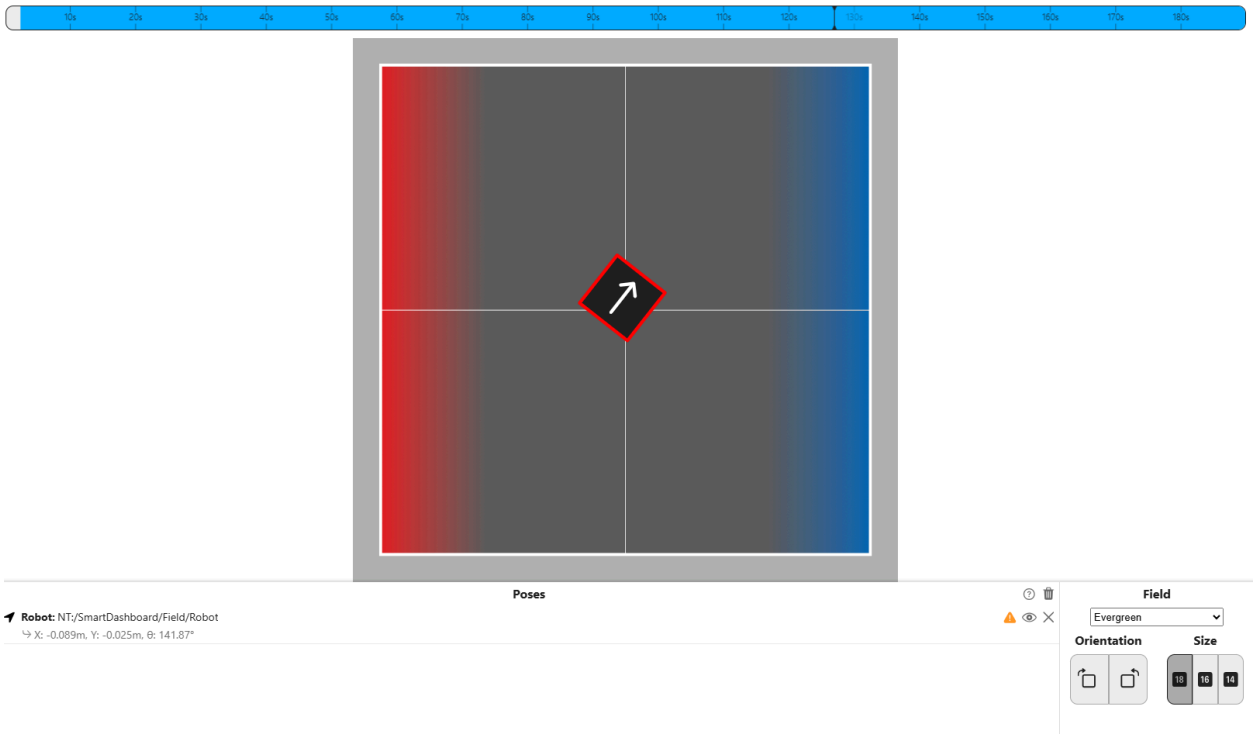
  // Update robot odometry
  double degrees = getGyroRotation();
  double leftMeters = Units.inchesToMeters(getLeftDistanceInch());
  double rightMeters = Units.inchesToMeters(getRightDistanceInch());
  odometry.update(Rotation2d.fromDegrees(degrees), leftMeters, rightMeters);
  Pose2d curPose = getPose();
  field.setRobotPose(curPose.getMeasureX(), curPose.getMeasureY(), curPose.getRotation());

  if(fastLogging || DataLogUtil.isMyLogRotation(logRotationKey)) {
    updateLog(logWhenDisabled: false);
  }
```

After this, save your code and simulate with advantage scope as a start tool. In advantage scope, find the field variable we sent to the smart dashboard. You may need to click ctrl + shift + k to reconnect. The field variable should look something like this.



If you did not start with a 2d field by default, click the plus sign in the top right corner and from the drop down menu, select 2d field. If you did start with a 2d field, ignore this step. Drag the robot variable from within the field into the poses section at the bottom of the 2d field window. Optionally, change the field to evergreen so that the romi is not placed on the much larger FRC field. Once this has been finished, Advantage Scope should look something like this.



After this, change the mode to teleop and drive your robot around. You should see the position of the robot update as you drive around, and the orientation update as you turn.

Being able to visualize the robot's position with advantage scope helps to find errors in the current path, and logging data in general can help to find errors and diagnose issues like excessive temperature, camera disconnects, etc. It is an essential part of programming, and highly useful.